

SLAMing im Labyrinth

Projektname: SLAMing im Labyrinth

Kurs: Einführung in die Robotik

Aufgabe 2 - 2024

Teammitglieder: Belal Kattan, Tjark Seemann

Betreuer: Eike Petersen

Ziel des Projekts

Das Ziel des Projekts "SLAMing im Labyrinth" besteht darin, einen Roboter zu entwickeln und zu programmieren, der:

1. Eine gegebene Labyrinthstruktur autonom kartografiert und dabei eine digitale Karte erstellt, die auf einem Bildschirm dargestellt wird.
2. Mithilfe der erstellten Karte autonom durch das Labyrinth navigiert, wobei farbige Felder (Rot, Grün, Blau) als Wegpunkte dienen.

Das Projekt erfordert die Umsetzung moderner Algorithmen zur Simultanen Lokalisierung und Kartenerstellung (SLAM) sowie eine robuste Integration von Sensorik und Steuerung für autonome Navigation.

Vorhandenes Grundgerüst

Das Grundgerüst des Projekts basiert auf einem bereitgestellten LegoOsc-Programm, das die Kommunikation zwischen einem Lego-Roboter und einer externen Software über das OSC-Protokoll ermöglicht. Dieses Programm wurde vom Betreuer zur Verfügung gestellt und umfasst die folgenden Hauptkomponenten:

1. Lego-Roboter:

- Ein programmierbarer Lego EV3-Roboter.
- Ausgestattet mit mehreren Sensoren:
 - Farbsensor
 - Ultraschallsensor
 - Gyroskopsensor
 - Berührungssensor
- Verwendete Motoren:
 - **Große Servomotoren:** Für die Hauptbewegung des Roboters (Antrieb der Räder).
 - **Mittlerer Servomotor:** Für präzisere Steueraufgaben, wie das Drehen eines Sensors.

2. OSC-Kommunikation:

- Ein Beispielprojekt namens "OscTester" wurde bereitgestellt, das OSC-Befehle an den Lego-Roboter senden kann.
- Dieses Projekt dient zur Validierung und Demonstration der OSC-Funktionalität.

Das Grundgerüst bietet eine solide Grundlage für die Implementierung der SLAM-Algorithmen und die Integration zusätzlicher Funktionen, die im Rahmen des Projekts erforderlich sind.

Idee und Modularität

Im Rahmen der Überlegungen und Planungen für das Projekt wurde ein modularer Ansatz entwickelt, um die vielfältigen Anforderungen systematisch zu bearbeiten. Wir haben beschlossen, dass ein Teammitglied die GUI- und Pathfinding-Algorithmus und der andere die Bewegungssteuerung und Automatisierung implementiert.

Bewegungssteuerung und Automatisierung

Dieses Modul umfasst folgende geplante Aufgaben:

- **Sensorintegration:**
 - Untersuchung und Einbindung des Farbsensors, Ultraschallsensors und Berührungssensors, um präzise Bewegungsdaten zu erhalten.
- **Motorsteuerung:**
 - Ausarbeitung einer Steuerlogik für die großen Servomotoren zur Hauptbewegung und den mittleren Servomotor für spezifische Aufgaben wie Drehbewegungen.
- **Bewegungsplanung:**
 - Entwurf und Optimierung von Algorithmen zur autonomen Navigation durch das Labyrinth. Dazu gehört das Umgehen von Hindernissen und das Erreichen farbiger Felder.
- **OSC-Integration:**
 - Planung und Umsetzung der OSC-Kommunikation zur Koordination von Bewegungs- und Sensordaten.

GUI und Pathfinding-Algorithmus

Die Aufgaben in diesem Modul werden wie folgt definiert:

- **Erstellung einer grafischen Benutzeroberfläche (GUI):**
 - Konzept und Entwicklung einer Oberfläche zur Visualisierung der vom Roboter erstellten Karte sowie des aktuellen Standorts und der geplanten Route.
 - **Pathfindung:**
 - Auswahl und Implementierung eines passenden Algorithmus (z. B. A* oder Dijkstra), um optimale Routen zwischen dem Startpunkt und den farbigen Feldern zu berechnen.
 - **Integration der Karte:**
 - Entwicklung einer Logik zur dynamischen Aktualisierung der Karte basierend auf den Sensordaten des Roboters.
 - **OSC-Kommunikation:**
 - Entgegennahme und Verarbeitung von Sensordaten sowie Übermittlung von Steuerbefehlen an das Bewegungsmodul.
-

Zusammenarbeit und Integration

Da die Aufgaben so klar getrennt sind, ist es wichtig, die Schnittstellen klar zu definieren.

Die wichtigsten dieser Schnittstellen sind:

Algorithmus -> Roboter-Bewegungssteuerung

Der Algorithmus muss die einzelnen Bewegungsrichtungen an die Steuerung schicken
Hier ist aber nur die richtige Reihenfolge wichtig.

Labyrinth-Status -> Gui

Die Informationen, welche das GUI benötigt, müssen in den Labyrinth Status eingetragen werden.

Der Informationsaustausch wird dann hier über einen Listener gewährleistet, der ein Event aufruft, sobald sich eine Information im Labyrinth ändert.

Labyrinth-Status -> Algorithmus

Da der Labyrinthstatus als Singleton existiert, kann der Algorithmus sich Informationen vom bekannten Labyrinth abrufen, sobald sie benötigt werden.

Bis auf diese Kommunikation zwischen den aufgabenbereichen haben wir versucht auf Kommunikation zwischen GUI/Algorithmus und Bewegung/Statussicherung

Diskussion zur Wahl des Pfadfindungsalgorithmus

Die Wahl des Pfadfindungsalgorithmus ist ein zentraler Bestandteil unseres Projekts. Ein effektiver Algorithmus ist entscheidend, um den Roboter präzise und effizient durch das Labyrinth zu navigieren. In dieser Diskussion werden verschiedene Algorithmen evaluiert und die endgültige Wahl begründet.

Anforderungen an den Algorithmus

Die Anforderungen an den Pfadfindungsalgorithmus sind:

1. **Effizienz:** Der Algorithmus sollte in der Lage sein, den optimalen Weg in kurzer Zeit zu berechnen.
2. **Robustheit:** Er muss in dynamischen und teilweise unbekannten Umgebungen zuverlässig funktionieren.
3. **Flexibilität:** Der Algorithmus sollte auf veränderte Bedingungen wie neue Hindernisse oder geänderte Start- und Zielpunkte reagieren können.
4. **Einfache Implementierung:** Die Integration des Algorithmus in unsere bestehende Software sollte mit vertretbarem Aufwand möglich sein.

Evaluierte Algorithmen

A*-Algorithmus

- **Vorteile:**
 - Berechnet den kürzesten Pfad effizient.
 - Heuristische Suche reduziert unnötige Berechnungen.
 - Gut geeignet für statische und dynamische Umgebungen.
- **Nachteile:**
 - Kann bei sehr großen Suchräumen ressourcenintensiv werden.

Dijkstra-Algorithmus

- **Vorteile:**
 - Liefert immer den kürzesten Pfad, unabhängig von der Heuristik.
 - Robuster gegenüber unvorhergesehenen Änderungen.
 - Ideal für kleinere oder begrenzte Suchräume.
- **Nachteile:**
 - Rechenaufwändiger bei großen Suchräumen als A*, da keine Heuristik verwendet wird.

Greedy-Best-First-Search

- **Vorteile:**
 - Sehr schnell in der Berechnung.
 - Konzentriert sich ausschließlich auf das Ziel.

- **Nachteile:**
 - Kann suboptimale Pfade liefern.
 - Nicht robust gegenüber Hindernissen oder dynamischen Änderungen.

Entscheidung

Nach der Evaluierung der Algorithmen wurde der **Dijkstra-Algorithmus** für unser Projekt ausgewählt. Diese Entscheidung basiert auf folgenden Überlegungen:

1. **Effizienz bei begrenztem Suchraum:** Das Labyrinth in unserem Szenario ist strukturell begrenzt und überschaubar. Der Dijkstra-Algorithmus kann hier seine Stärke ausspielen, da er nicht von einer Heuristik abhängt und dennoch den kürzesten Pfad liefert.
2. **Robustheit:** Der Algorithmus bietet eine verlässliche Berechnung unabhängig von dynamischen Änderungen oder unerwarteten Hindernissen.
3. **Einfache Implementierung:** Durch seine deterministische Arbeitsweise ist der Dijkstra-Algorithmus leichter in unser bestehendes System integrierbar und erfordert keine zusätzlichen Heuristik-Funktionen.

Während der A*-Algorithmus in größeren Suchräumen effizienter sein kann, ist der Dijkstra-Algorithmus in unserem spezifischen Fall besser geeignet, da das Labyrinth klar definiert und die Komplexität begrenzt ist. Die Möglichkeit, alle Knoten systematisch zu überprüfen, macht ihn besonders robust und ideal für unser Szenario.

Die Wahl des Dijkstra-Algorithmus gewährleistet eine präzise und zuverlässige Navigation durch das Labyrinth. Diese Entscheidung unterstützt unsere Zielsetzung, einen Roboter zu entwickeln, der sicher und effizient durch die Umgebung navigieren kann.

Planung des Pfadfindungsalgorithmus

Für die Navigation des Roboters im Projekt "SLAMing im Labyrinth" planen wir die Verwendung des Dijkstra-Algorithmus. Diese Planung konzentriert sich auf die theoretischen Grundlagen, die Modellierung des Labyrinths und die Anforderungen an den Algorithmus, ohne dabei die praktische Umsetzung zu beschreiben.

Ziel der Planung

Das Ziel dieser Planung ist es, einen klaren und detaillierten Ansatz für die Anwendung des Dijkstra-Algorithmus im Labyrinth zu definieren. Dabei wird das Labyrinth als Graph dargestellt, um die mathematischen Eigenschaften des Algorithmus optimal nutzen zu können. Dies stellt sicher, dass der Algorithmus effizient arbeitet und die Projektziele erreicht.

Mathematische Grundlage

Der **Dijkstra-Algorithmus** dient dazu, in einem **gewichteten Graphen** den kürzesten Pfad von einem Startknoten zu allen anderen Knoten zu bestimmen. Im Kontext unseres Labyrinths repräsentiert **jeder Knoten** eine Zelle, während **Kanten** die Verbindungen zwischen benachbarten Zellen sind. Das Kantengewicht drückt die Distanz oder Kosten für den Übergang von einer Zelle zur nächsten aus.

- **Knoten (V)**: Alle Zellen des Labyrinths.
- **Kanten (E)**: Verbindungen zwischen zwei benachbarten Zellen.
- **Gewicht $w(u, v)$** : Nichtnegative Distanz oder „Kosten“ für den Schritt von Zelle u nach Zelle v .

Schritte des Algorithmus

1. **Initialisierung**
 - Lege fest, dass die **Distanz** des Startknotens (s) **0** ist.
 - Alle anderen Knoten ($v \neq s$) erhalten zunächst eine **Distanz** von **unendlich**.
2. **Auswahl des Knotens mit geringster Distanz**
 - Solange es noch unbesuchte Knoten gibt, wähle immer jenen Knoten u aus, dessen bekannte Distanz aktuell **am kleinsten** ist.
 - Markiere diesen Knoten u als **besucht**, damit er nicht erneut ausgewählt wird.
3. **Relaxation (Entspannung) der Nachbarn**
 - Betrachte alle **Nachbarn** v des soeben besuchten Knotens u .
 - Prüfe, ob der **Weg über u** eine kleinere Distanz für v ergibt als der bisher bekannte Wert. Falls ja, aktualisiere Distanz(v) und speichere u als **Vorgänger** für v .
4. **Pfadrekonstruktion**
 - Sind alle Knoten abgearbeitet, kann für jeden Knoten v der kürzeste Pfad vom Startknoten s rekonstruiert werden, indem man den **gespeicherten Vorgängern** rückwärts folgt (von v zurück bis s).

Auf diese Weise liefert der Dijkstra-Algorithmus den kürzesten Pfad in jedem Labyrinth, solange alle Kantengewichte nicht negativ sind.

Anforderungen an den Algorithmus

- **Effizienz:** Der Algorithmus sollte in der Lage sein, die kürzesten Pfade in akzeptabler Zeit zu berechnen.
- **Robustheit:** Er muss mit unvollständigen oder dynamischen Graphen arbeiten können.
- **Klarheit:** Die Modellierung des Labyrinths sollte logisch nachvollziehbar sein.

Modellierung des Labyrinths

Graphenstruktur

- **Knoten:** Jede Zelle des Labyrinths wird durch einen eindeutigen Knoten dargestellt.
- **Kanten:** Verbindungen zwischen benachbarten Zellen werden durch Kanten repräsentiert. Das Gewicht der Kante basiert auf der Distanz zwischen den Zellen (z. B. die Länge der Verbindung).

Gewichtung

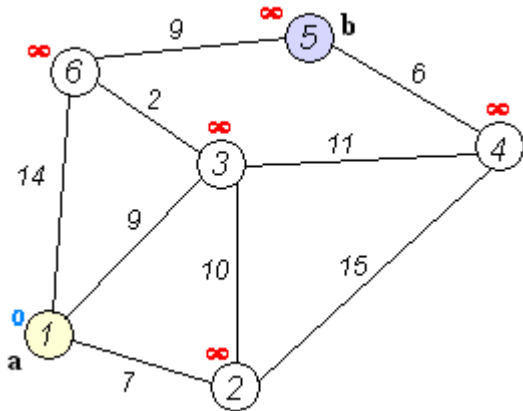
- Die Kanten werden durch physikalische Distanzen gewichtet.
- Hindernisse werden durch unendlich große Gewichte simuliert.

Überlegungen zur Implementierung

- **Start- und Zielpunkte:** Der Algorithmus wird für jede Navigationsanfrage mit einem definierten Start- und Zielknoten ausgeführt.
- **Dynamische Anpassung:** Änderungen im Labyrinth (z. B. entdeckte Hindernisse) erfordern eine Aktualisierung des Graphen.

Visualisierung der Planung

Schematische Darstellung des Algorithmus:



Mit dieser Planung legen wir den Grundstein für eine effiziente und robuste Navigation im Labyrinth. Der Dijkstra-Algorithmus wurde aufgrund seiner mathematischen Klarheit und Robustheit gewählt und passt ideal zu den Anforderungen unseres Projekts.

Beschreibung der GUI und der Pfadfindung (Simulation)

Die folgende Beschreibung basiert auf der alten Version des Projekts, vor der Integration der echten Bewegungssteuerung. Der Fokus liegt auf der grafischen Benutzeroberfläche (GUI) und der Simulation des Pfadfindungsalgorithmus. Am Ende wird erläutert, welche Klassen in der endgültigen Version weggelassen oder zusammengeführt werden.

Grafische Benutzeroberfläche (GUI)

Die GUI ist in der Klasse `MainGUI` implementiert und dient als zentrale Schnittstelle zwischen Benutzer und System. Sie umfasst mehrere Komponenten, die zusammenarbeiten, um das Labyrinth zu visualisieren und die Pfadfindung zu simulieren.

1. Hauptfenster (`MainGUI`)

- **Beschreibung:**
 - Das Hauptfenster stellt das Labyrinth als Karte dar und bietet interaktive Steuerelemente.
 - Es bindet mehrere Services wie `LabyrinthStateService` und `RobotStateService` ein, die den Zustand des Labyrinths und des Roboters verwalten.
- **Funktionen:**
 - Hinzufügen und Aktualisieren von Zellen im Labyrinth.
 - Steuerung der Simulation durch Tasteneingaben (z. B. Bewegung, Scannen, Exportieren und Laden der Karte).

2. Kartenanzeige (MapCanvas)

- **Beschreibung:**
 - Diese Komponente ist für die visuelle Darstellung des Labyrinths zuständig.
 - Zellen werden entsprechend ihrem Zustand (z. B. frei, blockiert, Farbfeld) eingefärbt.
- **Details zur Implementierung:**
 - Zeichnet Zellen basierend auf ihren Eigenschaften:
 - **Farbe:**
 - Weiß: Freie Zellen
 - Grau: Hindernisse
 - Rot, Grün, Blau: Farbzellen
 - Gelber Rahmen: Eingang
 - Rotes „X“: Blockierte Zellen
 - Zeichnet die Position und Richtung des Roboters als Dreieck.
- **Methoden:**
 - `addCell(x: Int, y: Int, cell: Cell)`: Fügt eine Zelle hinzu oder aktualisiert sie.
 - `updateRobotPosition(x: Int, y: Int)`: Aktualisiert die Position des Roboters.
 - `updateRobotDirection(direction: RoboterDirection)`: Aktualisiert die Ausrichtung des Roboters.

3. Steuerung und Interaktion

- **Beschreibung:**
 - Benutzerinteraktionen werden durch Tastendrucke ausgelöst und von `KeyListener`-Methoden in `MainGUI` verarbeitet.
- **Funktionen:**
 - **Bewegung:** Tasten `W`, `A`, `S`, `D` bewegen den Roboter in die vier Himmelsrichtungen.
 - **Scannen:** Mit der Eingabetaste (`ENTER`) wird die aktuelle Zelle gescannt.
 - **Farbzellen:**
 - `R`, `G`, `B` setzen die aktuelle Zelle als Farbzelle in Rot, Grün oder Blau.
 - **Blockierung:** `X` markiert eine Zelle als blockiert.
 - **Export und Import:**
 - `P` exportiert die Karte als JSON-Datei.
 - `L` lädt eine gespeicherte Karte.
 - **Pfadsimulation:**
 - `F` berechnet und visualisiert den kürzesten Pfad zu einer Farbzelle.

Pfadfindung (Simulation)

Die Pfadfindung basiert auf dem Dijkstra-Algorithmus und wird über den `LabyrinthStateService` bereitgestellt. Der Algorithmus berechnet den kürzesten Pfad von der aktuellen Position des Roboters zu einer Farbzelle.

1. Initialisierung des Graphen (`LabyrinthStateService`)

- **Beschreibung:**
 - Das Labyrinth wird als gewichteter Graph modelliert.
 - Knoten repräsentieren Zellen, Kanten die Verbindungen zwischen benachbarten Zellen.
- **Details:**
 - Blockierte Zellen werden durch das Entfernen ihrer Kanten dargestellt.
 - Farbzellen werden mit Prioritäten versehen, um sie gezielt anzusteuern.

2. Pfadberechnung

- **Methode:** `findPathDijkstra(start: Pair<Int, Int>, target: Pair<Int, Int>)`
- **Ablauf:**
 - Initialisiert Distanzen für alle Knoten.
 - Führt iterative Relaxation der Nachbarknoten durch.
 - Verwendet eine Priority Queue, um den nächsten Knoten mit der geringsten Distanz zu finden.
 - Rekonstruiert den Pfad durch Rückverfolgung der Vorgänger.

3. Visualisierung des Pfades

- **Beschreibung:**
 - Der berechnete Pfad wird in `MapCanvas` als cyanfarbige Linie dargestellt.
 - Methode: `calculateAndMoveToNextTarget` in `MainGUI`:
 - Markiert die Knoten des Pfades als cyan.
 - Zeichnet den Pfad dynamisch neu.

4. Simulierte Bewegung des Roboters

- **Beschreibung:**
 - Der Roboter folgt dem berechneten Pfad, wobei jede Bewegung in der GUI aktualisiert wird.
 - Methode: `moveRobotOneStep` in `MainGUI`:
 - Bewegt den Roboter schrittweise entlang des Pfades.
 - Aktualisiert die GUI nach jedem Schritt.

Klassenstruktur und zukünftige Änderungen

Die folgenden Klassen wurden in dieser Version verwendet:

1. **MainGUI**
 - Hauptklasse für die GUI und Benutzerinteraktion.
2. **MapCanvas**
 - Zuständig für die visuelle Darstellung des Labyrinths.
3. **LabyrinthStateService**
 - Verwaltet den Zustand des Labyrinths und implementiert die Pfadfindung.
4. **RobotStateService**
 - Verfolgt die Position und Ausrichtung des Roboters.
5. **MapController**
 - Ermöglicht das Hinzufügen und Aktualisieren von Zellen.

Zukünftige Änderungen

In der endgültigen Version werden einige Klassen entfernt oder in andere Klassen integriert:

- **MapController**-Funktionen werden direkt in **LabyrinthStateService** integriert.
- Simulationsmethoden wie **moveRobotOneStep** werden modifiziert zugunsten der tatsächlichen Bewegungssteuerung.
- GUI-Elemente wie **KeyListener** werden ersetzt durch Buttons und angepasst, um reale Steuerungen zu unterstützen.

Durch diese Anpassungen wird die Struktur des Systems optimiert und an die Anforderungen der echten Bewegungssteuerung angepasst.

Entwicklung der Labyrinth-Erkundung

In diesem Teil wird der Entwicklungsablauf zur automatischen Bewegung und Labyrinth Erkunden genauer erläutert

Simple Roboterbewegung

Als erstes wurden einfache Roboterbewegung implementiert, welche direkt über OSC-Übermittelt werden. Diese Bewegung soll unabhängig von der Roboterposition sein und universal anwendbar sein.

Diese Bewegung sind z.B Drehung nach links/rechts um 90 Grad, drehung um 180 grad, distanz Scan oder fahrt in die nächste Zelle

In diesen Funktionen werden die einzelnen OSC Befehle aufgerufen, um die bestimmte Bewegung abzuschließen.

Mit diesen Bewegungen kann man zwar den Roboter in der Theorie vollständig im Labyrinth bewegen, aber es werden noch keine Informationen über die Roboterposition und Ausrichtung im Labyrinth gespeichert und es wird auch nicht auf die Antworten des Roboters auf die Befehle reagiert.

Es fehlen also zwei wichtige Teile um Komplexere Bewegungen zu ermöglichen

1. Einen Movement-Manager welcher die nächsten OSC befehle schickt sobald einer abgeschlossen ist
2. Einen Moment-Planer welcher es ermöglicht Roboterbewegung leichter aufzurufen da Roboter Orientierung im Labyrinth mit berücksichtigt werden

Movement-Manager

Um mehrere OSC Befehle hintereinander abzuschicken um damit komplexere Bewegung zu ermöglichen und auch zu gewährleisten, dass nicht mehrere Befehle gleichzeitig abgeschickt werden.

Der Movement-Manager verfügt über eine Queue von Funktionen welche nacheinander aufgerufen werden und immer wenn eine OSC-Nachricht empfangen wird welche, das erfolgreiche Ausführen des Roboters bestätigt, wird der nächste Befehl aus der Queue herausgenommen und ausgeführt.

Movement-Planer

Da bis hierhin aber nur Bewegungen des Roboters aneinandergereiht werden können ohne einen Kontext zum Labyrinth herzustellen, plant der Movement-Planer nun, welche Roboter befehle in welcher Reihenfolge ausgeführt werden müssen, um zum Beispiel nach Norden zu fahren. Hier wird dann die Aktuelle Roboter Orientierung mit der gewünschten Richtung verglichen und die daraus folgenden OSC Befehle werden als Liste an den Movement-Manager übergeben.

Der nächste Schritt war nun, drei Teile zu testen und den Roboter per Knopfdrücke zu testen. Dies ist nun nicht mehr schwierig, da nur der Movement-Planer mit der gewünschten Richtung aufgerufen werden muss oder einfache Logik für das Abscannen einer Zelle implementiert werden muss.

Der nächste Schritt ist nun das Ablaufen des Labyrinths automatisch zu machen und eine automatische Speicherung des Labyrinths zu ermöglichen.

Automatische Erkundung und Speicherung

Da in der Bewegung der Knöpfe schon die Orientierung des Roboters mit enthalten ist, ist der Schritt zum automatischen Erkunden nicht mehr weit.

Es werden zwei Klassen benötigt, eine die die Logik beinhaltet, was in der Zelle für Aktionen ausgeführt wird, um die benötigten Informationen zu erhalten, wie z.B. welche Wände aus der Zelle kenne ich noch nicht? Drehe die Augen in die Richtung und scanne die Distanz.

Und eine Klasse namens Automatic-Explorer wessen Aufgabe ist zu berechnen, welche Zelle als nächstes erkundet werden soll.

Update des Labyrinths

Die gewonnenen Informationen werden direkt im OSC Handler in den Labyrinth-State eingetragen, dadurch wird der OSC-Handler sehr umfangreich und nicht mehr leicht verständlich.

Architektur und Besonderheiten

Durch das Verwenden der Movement-Queue und der Listener ist die Bewegung komplett Event-Driven und es kann auf das Benutzen von Timern im Ablauf des Absendens der OSC-Befehle verzichtet werden. Dadurch ist es auch einfach, neue Befehle und den Befehlsablauf zu ändern, da nur der Ablauf beim Hinzufügen in die Queue geändert werden muss.

Probleme bei der Implementierung der Bewegung

1. Update der Roboter Richtung:
Da wir die Berechnung und Ausführung der Befehle des Roboters Asynchron machen, ist die Orientierung des Roboters im Roboter-Status nicht übereinstimmend mit der realen Roboter-Ausrichtung. Der update der Roboter Richtung war nicht klar definiert und hat häufiger zu Problemen geführt und es hätte eine Roboter ausrichtung Systemintern und Roboter ausrichtung real geben müssen um das darstellen auf dem UI einfacher zu machen
 2. Verlorene OSC-Pakete:
Eines der größten Probleme war es, dass wir OSC-Pakete verlieren, hier war die Richtung des OSC-Pakets egal.
Die Lösung ist das Speichern des letzten OSC-Befehls und eine Timer von 5 sec wenn kein OSC-Paket eintrifft wird der letzte Befehl erneut abgesendet das geschieht so oft, bis ein OSC-Signal ankommt.
-

Endstand der GUI-Funktionalität (Überblick und Erläuterung)

Die grafische Benutzeroberfläche (GUI) teilt sich im Wesentlichen in zwei Hauptkomponenten auf:

1. **Das ControlPanel**
2. **Das MapCanvas**

Diese Komponenten werden wiederum in einem Hauptfenster, der **MainGUI**, mit Hilfe eines **GridBagLayout** angeordnet. Über Koin werden die notwendigen Services (z.B. **LabyrinthExplorer**, **AutomaticExplorer**, **LabyrinthStateService**, **Algorithmus** etc.) injiziert.

1. **ControlPanel** (Steuerungsbereich & Zellinformationen)

1.1 Zellinformationen

- **Textbereich (**cellInfo**):**
Im oberen Abschnitt des Panels befindet sich ein Textfeld (**JTextArea**) samt Scrollpane, das Informationen zur aktuell ausgewählten Zelle anzeigt. Angezeigt werden dabei:
 1. **Position** (*Koordinaten*)
 2. **Farbe** (z.B. *RED, GREEN, BLUE* etc.)
 3. **Blockiert** (*Boolean: true/false*)
 4. **Eingang** (*Boolean: true/false*)
 5. **Priorität** (*Integer*)
 6. **Wände** (*Angabe der Grenzen, z.B. NORTH=WALL, EAST=UNKNOWN* usw.)
- **Editierbarkeit:**
Das Textfeld ist editierbar. Nutzer können die dort aufgeführten Informationen manuell verändern.
- **„Speichern“-Button:**
Per Klick auf den Button wird **saveCellInfo()** aufgerufen.
Die Methode:
 1. Liest Zeile für Zeile den Text aus dem Feld.
 2. Aktualisiert oder erstellt bei Bedarf die entsprechende **Cell** im **labyrinthState**.
 3. Parsed dabei u.a. die Farbe (mittels **parseColor()**), den Blockiert-Status und die Wände.

4. Nach erfolgreicher Speicherung werden neue bzw. geänderte Zellinformationen in der Datenstruktur des `LabyrinthStateService` übernommen.

1.2 Steuerungs-Buttons

Das `ControlPanel` enthält diverse Buttons, die in einem `GridLayout` angeordnet sind. Diese Buttons decken wesentliche Bedienfunktionen ab:

1. **Navigation:**
 -  **Nach Norden**
 -  **Nach Süden**
 -  **Nach Westen**
 -  **Nach Osten**
→ `\rightarrow` Aufruf von `labyrinthExplorer.drive...()` zum Steuern des Roboters in die jeweilige Richtung.
2. **Zelle scannen** („ Zelle scannen“):
 - Startet den Scan-Vorgang der aktuellen Zelle via `labyrinthExplorer.scanCell()`.
 - Erkennt u.a. Farbflächen, Hindernisse oder Wände.
3. **Automatische Erkundung** („ Automatisch erkunden“):
 - Löst via `automaticExplorer.exploreAutomatic()` das autonome Befahren/Scannen des Labyrinths aus.
4. **Karte exportieren** („ Karte exportieren“):
 - Export der Labyrinth-Datenstruktur mittels `MapLoader.exportMap(labyrinthState, "labyrinth.json")`.
 - Erzeugt eine JSON-Datei, in der alle bekannten Zellen und deren Eigenschaften abgespeichert werden.
5. **Karte laden** („ Karte laden“):
 - Lädt eine zuvor exportierte Karte per `MapLoader.loadMap(labyrinthState, "labyrinth.json")`.
 - Aktualisiert daraufhin das interne `LabyrinthStateService` mit den geladenen Zellinformationen.
6. **Nächstes Ziel finden** („ Nächstes Ziel finden“):
 - Ruft `algorithmus.calculateAndMoveToNextTarget(labyrinthState)` auf, um bspw. den nächsten „interessanten“ Punkt (z.B. unerkundete Farbplatte oder spezielles Ziel) anzusteuern.
7. **Zurücksetzen** („ Zurücksetzen“):
 - Führt ein `labyrinthExplorer.resetRoboter()` aus.
 - Setzt damit die Roboterposition und ggf. weitere Zustände zurück.

2. MapCanvas (Visuelle Darstellung des Labyrinths)

2.1 Allgemeine Darstellung

- Das `MapCanvas` erbt von `JPanel` und implementiert `LabyrinthStateListener` und `RoboterStateListener`. Es reagiert damit auf Zustandsänderungen innerhalb des `LabyrinthStateService` und des `RobotStateService`.
- Die **Zellgröße** ist fest auf 30×30 Pixel gesetzt, das `MapCanvas` selbst ist für maximal 20×20 Zellen ausgelegt (bzw. $30 \times 20 = 600$ Pixel in der Breite und Höhe als grobe Orientierung).

2.2 Zeichnungslogik

- **Raster:**
Zunächst zeichnet das Canvas ein hellgraues Gitter für alle Zellen (0–20 in x- und y-Richtung). Hierdurch werden unbekannte bzw. nicht weiter definierte Zellen erkennbar.
- **Bekannte Zellen:**
Für jede im `labyrinthState` hinterlegte `Cell` werden:
 1. **Zellenfarbe** gefüllt (z.B. `Color.BLUE`, `Color.RED` etc.).
 2. **Wände** als schwarze Balken entlang des Zellenrandes gezeichnet, falls `cell.getBorder(...) == CellBoarder.WALL`.
 3. **Eingang** (falls `cell.isEntrance == true`) mit einem zusätzlichen gelben Rahmen hervorgehoben.
 4. **Blockierte Zellen** (falls `cell.isBlocked == true`) mit einem roten „X“ markiert (zwei diagonale Linien und ein „X“-Text).

2.3 Interaktion (Mausklick)

- Ein **Mausklick** auf eine Zelle (x, y) ruft den hinterlegten Listener auf (`selectedCellListener`).
- Dieser Listener wird in `MainGUI` an das `ControlPanel` gekoppelt: Sobald eine Zelle geklickt wird, zeigt das `ControlPanel` via `updateCellInfo(...)` die entsprechenden Daten im Textfeld an.

2.4 Roboter-Darstellung

- Die aktuelle **Roboterposition** wird vom `RobotStateService` abgefragt.
- Darstellung erfolgt als **magentafarbenes Dreieck** im Mittelpunkt der jeweiligen Zelle.
- (Optional) Je nach Implementierung könnte die Dreiecks-Ausrichtung (`NORTH`, `EAST`, `SOUTH`, `WEST`) dynamisch sein. Im Code-Beispiel wird exemplarisch mit `RoboterDirection.NORTH` gezeichnet, kann jedoch um die reale Roboter-Ausrichtung erweitert werden.

3. MainGUI (Rahmen und Zusammenführung)

- **MainGUI** erbt von **JFrame** und richtet mittels **GridBagLayout** die beiden Hauptkomponenten an:
 1. **MapCanvas** (grafische Labyrinthdarstellung)
 2. **ControlPanel** (Steuerung und Zellinformation)
 - Ein **Titel-Label** („Peter der tolle Roboter“) wird über den beiden Bereichen zentral angezeigt.
 - Nach dem Anordnen aller Elemente wird das Fenster mittels **pack()** auf eine angemessene Größe zusammengefasst und mit **isVisible = true** sichtbar gemacht.
-

Zusammenfassung der GUI-Fähigkeiten

1. **Interaktive Labyrinthansicht:**
 - Gitterdarstellung (unbekannte Bereiche als graue Felder).
 - Farbige Markierung bekannter Zellen inkl. Wände, Eintrittszellen, Blockaden.
 - Roboter-Position als Dreieck.
2. **Zell-Inspektion und -Bearbeitung:**
 - Per Klick auf eine Zelle werden alle Informationen angezeigt und können manuell im Textbereich geändert werden.
 - Ein Klick auf „Speichern“ ermöglicht die direkte Aktualisierung im **LabyrinthStateService**.
3. **Roboter-Steuerung:**
 - Manuelle Navigation (N, S, W, O).
 - Zellen scannen.
 - Automatische Erkundung starten.
4. **Kartenverwaltung:**
 - Export (JSON) und Import bestehender Kartendaten.
 - Zur Laufzeit können so Labyrinthzustände geladen oder gesichert werden.
5. **Zielsuche:**
 - Automatisierte Berechnung/Ansteuerung eines nächsten „Ziels“ (z.B. unerkundete Bereiche).
6. **Zurücksetzen:**
 - Roboter- und Labyrinthzustand auf Anfangswerte zurücksetzen.

Die GUI bietet somit alle relevanten Werkzeuge, um das (modular aufgebaute) Labyrinth interaktiv zu kartieren, zu erkunden und den Roboter darin zu steuern. Gleichzeitig erlaubt sie eine flexible Datenerfassung und -manipulation (Farbe, Wände, Blockierung etc.) und unterstützt den Nutzer bei der späteren Auswertung oder Wiederverwendung der Labyrinthzustände per JSON-Export/-Import.

Probleme bei der Vorstellung und ihre Lösung

Während der Vorstellung des Projekts "SLAMing im Labyrinth" traten zwei zentrale Probleme auf, die sowohl den Datenbestand als auch den Ablauf des Pfadfindung Algorithmus beeinflussten. Im Folgenden werden diese Probleme detailliert dargestellt und die jeweiligen Lösungen erläutert.

1. Überschreiben der bestehenden Kartendatei

Problem: Beim Exportieren des Labyrinths (`labyrinth.json`) wurde die bereits existierende Datei immer wieder überschrieben. Dies führte dazu, dass ältere Kartenversionen verloren gingen und somit wertvolle Daten und Zwischenstände nicht mehr verfügbar waren.

Ursache: In der ursprünglichen Implementierung legte die Exportfunktion den neuen Datenstand immer unter demselben Dateinamen ab. Dadurch wurde der bisherige Inhalt ohne Rückfrage ersetzt.

Konsequenz: Sobald eine Karte neu exportiert wurde, ging die zuvor gespeicherte Version komplett verloren.

Lösung: Um ältere Versionen zu bewahren, wurde der Exportmechanismus so angepasst, dass jeder neue Export in einer eigenständigen Datei gespeichert wird. Konkret wird dem ursprünglichen Dateinamen automatisch ein Index (`_1`, `_2`, `_3`, ...) angehängt. Auf diese Weise werden ältere Versionen nicht überschrieben, sondern bleiben als eigenständige Datei erhalten. Dies ermöglicht eine lückenlose Dokumentation verschiedener Projektstände und erleichtert das Wiederherstellen früherer Zustände.

2. Konflikte mit dem Pfadfindungsalgorithmus bei echter Roboterbewegung

Problem: Bei der Umstellung von einer rein simulativen Bewegung (mittels Timer) auf echte Bewegungsbefehle für den Roboter traten unerwartete Probleme im Pfadfindungsalgorithmus auf. Durch die geänderte Logik – insbesondere das rekursive Neuberechnen neuer Ziele nach jedem Schritt – konnte es zu einer Endlosschleife im Code kommen. Diese Endlosschleife rief immer wieder die Methode `calculateAndMoveToNextTarget` auf und führte letztlich zu einer `StackOverflowError`-Exception:

Exception in thread "AWT-EventQueue-0" java.lang.StackOverflowError

```
    at de.fhkiel.rob.legoosctester.Algorithmus.findPathDijkstra(Algorithmus.kt:16)
    at
de.fhkiel.rob.legoosctester.Algorithmus.calculateAndMoveToNextTarget(Algorithmus.kt:110)
    at
de.fhkiel.rob.legoosctester.Algorithmus.calculateAndMoveToNextTarget(Algorithmus.kt:154)
```

Ursache: In der alten, simulationsbasierten Version wurden Pfade schrittweise über einen Timer abgefahren. In der neuen Version rief die Methode jedoch am Ende jedes abgelaufenen Pfads erneut sich selbst auf, ohne vor dem nächsten Zielaufwurf zu prüfen, ob ein neues Ziel tatsächlich existiert oder ob die Situation dies erforderte. Dies resultierte in einer unendlichen Rekursion.

Konsequenz: Sobald das System in diesen rekursiven Aufruf geriet, belegte der Code immer mehr Platz auf dem Aufrufstack, bis die `StackOverflowError` ausgelöst wurde. Der Roboter konnte infolgedessen weder seine Pfade korrekt abarbeiten noch stabil weiter navigieren.

Lösung: Die Methode, welche die Pfadfindung wiederholt aufrief, wurde umstrukturiert. Anstelle einer direkten rekursiven Selbstaufruf-Kette wurde eine Schleifen- und Aufruflogik implementiert, die vor jedem erneuten Start einer Pfadberechnung verifiziert, ob tatsächlich ein neues Ziel vorhanden ist und angefahren werden soll. Damit konnte das Problem der Endlosschleife behoben werden. Nach dieser Anpassung verarbeitet das System nun in geordneter Reihenfolge alle Farbzellen sowie den anschließenden Rückweg zum Eingang, ohne in eine endlose Rekursion zu geraten.

Durch das Beheben dieser beiden Kernprobleme konnte einerseits der Verlust älterer Kartenversionen verhindert werden, andererseits wurde der Pfadfindungsalgorithmus stabil an die echte Roboterbewegung angepasst. Dies bildet die Grundlage für eine zuverlässige autonome Navigation und eine lückenlose Projektdokumentation.

